

系统开发工具基础

第二周实验报告

姓名：林子钰

学号：25060012017 班级：计算机类1班

2026年8月31日

目录

1	实验概述	3
2	实验环境	3
3	课堂实验	3
3.1	实验五：控制一个可清理的后台任务	3
3.1.1	实验目的	3
3.1.2	关键过程	3
3.2	创建并运行Shell脚本	3
3.3	暂停前台任务并放入后台	4
3.4	获取进程号	4
3.5	发送终止信号	4
3.6	使用trap执行清理操作	4
3.7	检查实验结果	5
3.7.1	实验结果	5
3.7.2	问题分析与体会	6
3.8	实验六：使用 Ruff 和 pytest 完善 Python 项目	6
3.9	实验七：使用 pdb、pytest 和 Git 调试归并排序程序	14
3.10	实验八：使用 cProfile 分析并优化 Python 程序性能	18
4	课后练习	24
5	课后练习	26
5.1	练习1：ls -l	26

5.2	练习2: Glob 通配模式	26
5.3	练习3: 使用命令连接符控制执行顺序	27
5.4	练习4: 创建和使用 Shell 别名	27
5.5	练习5: 统计历史命令的使用频率	28
5.6	练习6: 使用 Git 管理配置文件	29
5.7	练习7: 自定义 Shell 提示符	31
5.8	练习8: 编写 dotfiles 自动安装脚本	32
6	GitHub/Gitee 仓库与提交记录	33
7	实验总结	33
8	GitHub/Gitee 仓库与提交记录	33
9	实验总结	33

1 实验概述

第二周实验主要围绕程序调试与性能分析展开，重点学习如何根据错误信息、调用栈和变量状态定位程序问题，并通过测试验证修复结果。

本周实验涉及语法错误、运行时错误和逻辑错误的排查，同时练习使用断点、单步执行、变量监视等调试方法。在归并排序实验中，需要重点检查循环条件和数组下标，例如将下标 j 错写为 i 可能导致排序结果错误。此外，还需要使用计时或性能分析工具查找程序运行瓶颈，对比优化前后的执行效率。

通过本周实验，我掌握了基本的程序调试和性能分析方法，理解了先保证程序正确、再进行性能优化的重要性，为后续深度学习和科研项目中的问题排查打下基础。

2 实验环境

表 1: 实验环境

项目	配置
操作系统	Ubuntu 24.04.4
Shell	Bash
Git 版本	2.34.1
LaTeX 环境	pdfTeX 3.141592653-2.6-1.40.22 (TeX Live 2022/dev/Debian)
编译引擎	XeLaTeX
编辑器	GNU nano, version 6.2

3 课堂实验

3.1 实验五：控制一个可清理的后台任务

3.1.1 实验目的

1.掌握Shell前台任务与后台任务的基本概念。2.掌握使用 `Ctrl-Z` 暂停前台进程、使用 `bg` 将任务放入后台运行的方法。3.掌握使用 `jobs` 查看Shell作业、使用 `jobs -p` 获取进程号的方法。4.理解Linux信号机制，能够使用 `kill -TERM` 向进程发送终止信号。5.掌握Shell脚本中的 `trap` 命令，理解程序退出时执行清理操作的机制。

3.1.2 关键过程

3.2 创建并运行Shell脚本

首先创建实验脚本并赋予执行权限：

```
1 touch worker.sh
2 chmod +x worker.sh
3 bash -n worker.sh
4 ./worker.sh
```

其中，`touch`用于创建脚本文件，`chmod +x`为脚本添加执行权限，`bash -n`用于检查脚本语法，`./worker.sh`用于运行脚本。

3.3 暂停前台任务并放入后台

脚本运行后，按下`Ctrl-Z`暂停当前前台任务，然后执行以下命令：

```
1 bg
2 jobs
```

`Ctrl-Z`会暂停前台进程，`bg`将暂停的任务放入后台继续运行，`jobs`用于查看当前Shell中的作业及其运行状态。

3.4 获取进程号

使用以下命令获取后台任务对应的进程号：

```
1 jobs -p
2 pid=$(jobs -p | head -n 1)
3 printf "PID=%s\n" "$pid"
```

`jobs -p`只输出作业对应的进程号。通过命令替换将进程号保存到变量`pid`中，便于后续发送信号。

3.5 发送终止信号

向后台进程发送`TERM`终止信号，并等待进程退出：

```
1 kill -TERM "$pid"
2 wait "$pid"
```

`kill -TERM`用于向指定进程发送正常终止信号，`wait`用于等待后台进程结束，避免Shell过早执行后续命令。

3.6 使用trap执行清理操作

在脚本中使用`trap`捕获终止信号：

```
1 trap 'echo CLEAN_EXIT >> cleanup.log; exit 0' TERM INT
```

当脚本接收到TERM或INT信号时，会将CLEAN_EXIT写入cleanup.log，然后正常退出。该机制可以用于程序结束前释放资源、保存日志等清理操作。

3.7 检查实验结果

最后检查清理日志内容：

```
1 cat cleanup.log
```

如果终端输出CLEAN_EXIT，说明脚本成功捕获信号并执行了退出清理操作。

3.7.1 实验结果

```
yu@Tokito:~/q05$ ./worker.sh > stdout.log 2> stderr.log
^Z
[1]+  Stopped                  ./worker.sh > stdout.log 2> stderr.log
yu@Tokito:~/q05$ bg
jobs
[1]+  ./worker.sh > stdout.log 2> stderr.log &
[1]+  Running                  ./worker.sh > stdout.log 2> stderr.log &
yu@Tokito:~/q05$ pid=$(jobs -p | head -n 1)
yu@Tokito:~/q05$ printf 'PID=%s\n' "$pid"
PID=2520
yu@Tokito:~/q05$ kill -TERM "$pid"
yu@Tokito:~/q05$ wait "$pid"
[1]+  Done                    ./worker.sh > stdout.log 2> stderr.log
yu@Tokito:~/q05$ grep -Fx 'CLEAN_EXIT' cleanup.log
CLEAN_EXIT
CLEAN_EXIT
yu@Tokito:~/q05$ test -z "$(jobs -p)" && echo 'job exited'
job exited
yu@Tokito:~/q05$ cat stdout.log
0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
yu@Tokito:~/q05$ cat stderr.log
yu@Tokito:~/q05$
```

图 1: Q05实验结果

图 1展示了Q05中Shell前后台任务切换、进程终止以及trap清理操作的关键运行结果。

实验结果表明，Shell脚本成功启动并在前台运行。按下Ctrl-Z后，前台任务被暂停，并显示为Stopped状态；执行bg命令后，该任务成功转入后台运行，jobs命令可以查看作业状态。

随后，通过jobs -p获取后台任务的进程号，例如PID=2520，并使用kill -TERM向该进程发送终止信号。wait命令等待后台任务结束，终端显示Done，说明进程已经正常退出。执行grep检查cleanup.log，成功输出CLEAN_EXIT，表明trap命令已经捕获终止信号并完成清理操作。最后，使用jobs -p检查作业列表为空，并输出job exited。

3.7.2 问题分析与体会

本题需要重点理解Shell作业与进程之间的关系。Ctrl-Z只能暂停当前前台任务，任务暂停后还需要使用bg才能让它后台继续运行；jobs用于查看当前Shell管理的作业，而jobs -p可以获取作业对应的进程号。

使用kill -TERM发送终止信号时，脚本不会简单地被强制结束，而是可以通过trap捕获信号，执行日志记录和资源清理等操作。因此，在程序退出前安排清理逻辑非常重要。实际操作中还应注意，wait的对象必须是有效的进程号，并且应在终止后台任务后检查cleanup.log和作业状态，从而确认信号处理和清理过程确实执行成功。

3.8 实验六：使用 Ruff 和 pytest 完善 Python 项目

实验目的：学习 Python 项目的基本组织方式，掌握函数编写、程序调用、单元测试、代码规范检查以及 Git 分阶段提交等开发工具的使用方法。

创建实验目录：

```
1 mkdir q06
2 cd q06
```

创建 Python 工具函数文件 math_utils.py:

```
1 printf 'def calculate_total(price: float, count: int) -> float:\n return price\n * count\n' > math_utils.py
```

该函数接收商品单价和数量两个参数，并返回商品总价。参数类型和返回值类型使用类型注解进行说明。

创建程序入口文件 app.py:

```
1 printf 'from math_utils import calculate_total\n\nprint(calculate_total(12.5,\n 4))\n' > app.py
```

运行程序：

```
1 python3 app.py
```

程序输出结果如下：

```
1 50.0
```

这说明程序能够正确导入 `calculate_total` 函数，并计算出单价为 12.5、数量为 4 时的商品总价。

创建测试文件 `test_math_utils.py`:

```
1 printf 'from math_utils import calculate_total\n\n\ndef test_calculate_total()
    :\n assert calculate_total(12.5, 4) == 50.0\n' > test_math_utils.py
```

运行 `pytest` 测试:

```
1 python3 -m pytest -q
```

首次运行测试时，如果系统尚未安装 `pytest`，终端会提示:

```
1 /usr/bin/python3: No module named pytest
```

此时更新软件源并安装 `pytest`:

```
1 sudo apt update
2 sudo apt install -y python3-pytest
```

安装完成后再次运行测试:

```
1 python3 -m pytest -q
```

测试结果如下:

```
1 1 passed in ...s
```

测试结果显示共有一个测试用例通过，说明 `calculate_total` 函数能够得到预期结果。安装 `Ruff` 并进行代码质量检查:

```
1 sudo snap install ruff
2 ruff --version
3 ruff check .
```

如果 `Ruff` 检测到代码格式或导入顺序问题，可以使用以下命令自动修复:

```
1 ruff check . --fix
2 ruff check .
```

修复完成后，检查结果如下:

```
1 All checks passed!
```

这表明当前项目中的 Python 文件已经通过 `Ruff` 的代码质量检查。

修改函数名称并检查项目:

```
1 sed -i 's/total_price/calculate_total/g' math_utils.py app.py test_math_utils.py
2 python3 app.py
3 python3 -m pytest -q
4 ruff check .
```

运行结果如下：

```
1 50.0
2 1 passed in ...s
3 All checks passed!
```

修改函数名称后，程序运行结果、单元测试结果和 Ruff 检查结果均正常。在修改过程中，如果终端提示语法错误，应根据错误提示检查对应文件，重点查看导入语句是否重复拼接、括号是否匹配以及缩进是否正确。

创建 Git 仓库并配置主分支：

```
1 git init
2 git branch -M main
```

创建 '.gitignore' 文件，忽略 Python 运行过程中自动生成的缓存目录和字节码文件：

```
1 printf '__pycache__/\n.pytest_cache/\n*.py[cod]\n' > .gitignore
```

查看当前文件状态：

```
1 git status
```

进行第一次分阶段提交，提交 '.gitignore' 文件：

```
1 git add .gitignore
2 git commit -m "chore: ignore Python cache files"
```

进行第二次提交，提交核心功能代码：

```
1 git add math_utils.py
2 git commit -m "feat: add calculate_total utility"
```

进行第三次提交，提交程序入口和测试代码：

```
1 git add app.py test_math_utils.py
2 git commit -m "test: add calculate_total usage and test"
```

本实验按照文件功能进行分阶段提交，没有将所有文件一次性提交。查看提交历史：


```
1 git log --oneline --graph --all
```

提交历史示例如下：

```
1 e665c9d (HEAD -> main) test: add calculate_total usage and test
2 6bca5b3 feat: add calculate_total utility
3 91367ac chore: ignore Python cache files
```

检查 Git 工作区状态：

```
1 git status
```

如果所有文件均已提交，终端会显示：

```
1 On branch main
2 nothing to commit, working tree clean
```

这说明当前 Git 工作区干净，没有未提交的修改。

配置并查看远程仓库：

```
1 git remote add origin https://github.com/Tokito-git/sdt_week2-25060012017.git
2 git remote -v
```

远程仓库地址如下：

```
1 https://github.com/Tokito-git/sdt_week2-25060012017.git
```

如果执行远程推送时出现连接被拒绝或网络超时，说明当前环境暂时无法连接 GitHub。此时本地代码编写、测试、代码规范检查和 Git 分阶段提交仍然已经完成；网络恢复后可以使用以下命令继续上传：

```
1 git push -u origin main
```

```
yu@Tokito:~/q06$ cd q06
yu@Tokito:~/q06$ cat > math_utils.py <<'EOF'
> def total_price(price: float, count: int) -> float:
>     return price * count
> EOF
yu@Tokito:~/q06$ cat > app.py <<'EOF'
from math_utils import total_price
>
> print(total_price(12.5, 4))
> EOF
yu@Tokito:~/q06$ cat > test_math_utils.py <<'EOF'
> from math_utils import total_price
>
> def test_total_price():
>     assert total_price(12.5, 4) == 50.0
> EOF
yu@Tokito:~/q06$ cd app.py
bash: cd: app.py: Not a directory
yu@Tokito:~/q06$ pwd
/home/yu/q06
yu@Tokito:~/q06$ ls -l
total 16
-rw-rw-r-- 1 yu yu 64 9月 2 13:18 app.py
-rw-rw-r-- 1 yu yu 77 9月 2 13:18 math_utils.py
drwxrwxr-x 2 yu yu 4096 8月 31 16:07 __pycache__
-rw-rw-r-- 1 yu yu 100 9月 2 13:19 test_math_utils.py
yu@Tokito:~/q06$ cat math_utils.py
def total_price(price: float, count: int) -> float:
    return price * count
yu@Tokito:~/q06$ cat app.py
from math_utils import total_price

print(total_price(12.5, 4))
yu@Tokito:~/q06$ cat test_math_utils.py
from math_utils import total_price

def test_total_price():
    assert total_price(12.5, 4) == 50.0
yu@Tokito:~/q06$ python3 app.py
50.0
yu@Tokito:~/q06$ python3 -m pytest -q
/usr/bin/python3: No module named pytest
```

图 2: 创建实验文件并检查 Python 代码

```
yu@Tokito:~/q06$ python3 -m pytest -q
/usr/bin/python3: No module named pytest
yu@Tokito:~/q06$ sudo apt update
[sudo] password for yu:
Hit:2 http://security.ubuntu.com/ubuntu jammy-security InRelease
Hit:1 http://mirrors.tuna.tsinghua.edu.cn/ubuntu jammy InRelease
Get:3 http://mirrors.tuna.tsinghua.edu.cn/ubuntu jammy-updates InRelease [128 kB]
Hit:4 http://mirrors.tuna.tsinghua.edu.cn/ubuntu jammy-backports InRelease
Get:5 http://mirrors.tuna.tsinghua.edu.cn/ubuntu jammy-updates/main amd64 Packages [3,775 kB]
Get:6 http://mirrors.tuna.tsinghua.edu.cn/ubuntu jammy-updates/main i386 Packages [1,073 kB]
Get:7 http://mirrors.tuna.tsinghua.edu.cn/ubuntu jammy-updates/universe i386 Packages [818 kB]
Get:8 http://mirrors.tuna.tsinghua.edu.cn/ubuntu jammy-updates/universe amd64 Packages [1,282 kB]
Fetched 7,076 kB in 12s (571 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
115 packages can be upgraded. Run 'apt list --upgradable' to see them.
yu@Tokito:~/q06$ sudo apt install -y python3-pytest
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  python3-attr python3-iniconfig python3-packaging python3-pluggy python3-py python3-pygments python3-toml
Suggested packages:
  python-attr-doc subversion python-pygments-doc ttf-bitstream-vera
The following NEW packages will be installed:
  python3-attr python3-iniconfig python3-packaging python3-pluggy python3-py python3-pygments python3-pytest
  python3-toml
0 upgraded, 8 newly installed, 0 to remove and 115 not upgraded.
Need to get 1,152 kB of archives.
After this operation, 6,107 kB of additional disk space will be used.
Get:1 http://mirrors.tuna.tsinghua.edu.cn/ubuntu jammy-updates/main amd64 python3-attr all 21.2.0-1ubuntu1 [43.9 kB]
Get:2 http://mirrors.tuna.tsinghua.edu.cn/ubuntu jammy/universe amd64 python3-iniconfig all 1.1.1-2 [6,024 B]
Get:3 http://mirrors.tuna.tsinghua.edu.cn/ubuntu jammy/main amd64 python3-packaging all 21.3-1 [30.7 kB]
Get:4 http://mirrors.tuna.tsinghua.edu.cn/ubuntu jammy/universe amd64 python3-pluggy all 0.13.0-7.1 [19.0 kB]
Get:5 http://mirrors.tuna.tsinghua.edu.cn/ubuntu jammy/universe amd64 python3-py all 1.10.0-1 [71.9 kB]
Get:6 http://cn.archive.ubuntu.com/ubuntu jammy-updates/main amd64 python3-pygments all 2.11.2+dfsg-2ubuntu0.1 [750 kB]
Get:7 http://mirrors.tuna.tsinghua.edu.cn/ubuntu jammy/universe amd64 python3-pytest all 6.2.5-1 [249 kB]
Get:8 http://mirrors.tuna.tsinghua.edu.cn/ubuntu jammy/universe amd64 python3-toml all 0.10.2-1 [14.2 kB]
Fetched 1,152 kB in 1s (1,152 kB/s)
debconf: delaying package configuration, since apt-utils is not installed
Selecting previously unselected package python3-attr.
(Reading database ... 123456789 files and directories currently installed.)
Preparing to unpack .../python3-attr_21.2.0-1ubuntu1_all.deb ...
Unpacking python3-attr (21.2.0-1ubuntu1) ...
Selecting previously unselected package python3-iniconfig.
Preparing to unpack .../python3-iniconfig_1.1.1-2_all.deb ...
Unpacking python3-iniconfig (1.1.1-2) ...
Selecting previously unselected package python3-packaging.
Preparing to unpack .../python3-packaging_21.3-1_all.deb ...
Unpacking python3-packaging (21.3-1) ...
Selecting previously unselected package python3-pluggy.
Preparing to unpack .../python3-pluggy_0.13.0-7.1_all.deb ...
Unpacking python3-pluggy (0.13.0-7.1) ...
Selecting previously unselected package python3-py.
Preparing to unpack .../python3-py_1.10.0-1_all.deb ...
Unpacking python3-py (1.10.0-1) ...
Selecting previously unselected package python3-pygments.
Preparing to unpack .../python3-pygments_2.11.2+dfsg-2ubuntu0.1_all.deb ...
Unpacking python3-pygments (2.11.2+dfsg-2ubuntu0.1) ...
Selecting previously unselected package python3-pytest.
Preparing to unpack .../python3-pytest_6.2.5-1_all.deb ...
Unpacking python3-pytest (6.2.5-1) ...
Selecting previously unselected package python3-toml.
Preparing to unpack .../python3-toml_0.10.2-1_all.deb ...
Unpacking python3-toml (0.10.2-1) ...
Setting up python3-attr (21.2.0-1ubuntu1) ...
Setting up python3-iniconfig (1.1.1-2) ...
Setting up python3-packaging (21.3-1) ...
Setting up python3-pluggy (0.13.0-7.1) ...
Setting up python3-py (1.10.0-1) ...
Setting up python3-pygments (2.11.2+dfsg-2ubuntu0.1) ...
Setting up python3-pytest (6.2.5-1) ...
Setting up python3-toml (0.10.2-1) ...
```

图 3: 更新软件源并安装 pytest

```
Setting up python3-toml (0.10.2-1) ...
Setting up python3-pluggy (0.13.0-7.1) ...
Setting up python3-pytest (6.2.5-1ubuntu2) ...
Processing triggers for man-db (2.10.2-1) ...
yu@Tokito:~/q06$ python3 -m pytest -q
.
1 passed in 0.03s [100%]
yu@Tokito:~/q06$ ruff check .
Command 'ruff' not found, but can be installed with:
sudo snap install ruff
yu@Tokito:~/q06$ python3 -m ruff check app.py
/usr/bin/python3: No module named ruff
yu@Tokito:~/q06$ sudo apt update
[sudo] password for yu:
Hit:3 http://security.ubuntu.com/ubuntu jammy-security InRelease
Hit:1 http://mirrors.tuna.tsinghua.edu.cn/ubuntu jammy InRelease
Hit:2 http://mirrors.tuna.tsinghua.edu.cn/ubuntu jammy-updates InRelease
Hit:4 http://mirrors.tuna.tsinghua.edu.cn/ubuntu jammy-backports InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
115 packages can be upgraded. Run 'apt list --upgradable' to see them.
yu@Tokito:~/q06$ sudo apt install -y ruff
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done

No apt package "ruff", but there is a snap with that name.
Try "snap install ruff"

E: Unable to locate package ruff
yu@Tokito:~/q06$ sudo snap install ruff
ruff 0.16.4 from Alex Lowe (lengau) installed
yu@Tokito:~/q06$ ruff --version
ruff 0.16.4
yu@Tokito:~/q06$ ruff check .
I001 [*] Import block is un-sorted or un-formatted
--> test_math_utils.py:1:1

1 | from math_utils import total_price
  | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
2 |
3 | def test_total_price():
  |
help: Organize imports
2 |
3 | +
4 | def test_total_price():

Found 1 error.
[*] 1 fixable with the '--fix' option.
yu@Tokito:~/q06$ ruff check . --fix
Found 1 error (1 fixed, 0 remaining).
yu@Tokito:~/q06$ cat test_math_utils.py
from math_utils import total_price

def test_total_price():
    assert total_price(12.5, 4) == 50.0
yu@Tokito:~/q06$ ruff check .
All checks passed!
```

图 4: 安装 Ruff 并修复代码格式问题

```

3 +
4 | def test_total_price():
5 |
Found 1 error.
[*] 1 fixable with the '--fix' option.
yu@Tokito:~/q00$ ruff check . --fix
Found 1 error (1 fixed, 0 remaining).
yu@Tokito:~/q00$ cat test_math_utils.py
from math_utils import total_price

def test_total_price():
    assert total_price(12.5, 4) == 50.0
yu@Tokito:~/q00$ ruff check .
All checks passed!
yu@Tokito:~/q00$ python3 -m pytest -q
.
1 passed in 0.05s
yu@Tokito:~/q00$ sed -i 's/total_price/calculate_total/g' math_utils.py app.py test_math_utils.py
yu@Tokito:~/q00$ grep -Rni 'total_price\\|calculate_total' .
./test_math_utils.py:1:from math_utils import calculate_total
./test_math_utils.py:4:def test_calculate_total():
./test_math_utils.py:5:    assert calculate_total(12.5, 4) == 50.0
./app.py:1:from math_utils import calculate_total
./app.py:3:print(calculate_total(12.5, 4))
./__pycache__/v/cache/nodeids:2: "test_math_utils.py::test_total_price"
./math_utils.py:1:def calculate_total(price: float, count: int) -> float:
grep: ./__pycache__/test_math_utils.cpython-310-pytest-6.2.5.pyc: binary file matches
grep: ./__pycache__/math_utils.cpython-310.pyc: binary file matches
yu@Tokito:~/q00$ python3 app.py
50.0
yu@Tokito:~/q00$ python3 -m pytest -q
.
1 passed in 0.04s
yu@Tokito:~/q00$ nano app.py
yu@Tokito:~/q00$ ruff check app.py
invalid-syntax: Simple statements must be separated by newlines or semicolons
--> app.py:5:32
3 | from math_utils import calculate_total
4 |
5 | print(calculate_total(12.5, 4))from math_utils import calculate_total
6 |
7 | print(calculate_total(12.5, 4))

Found 1 error.
yu@Tokito:~/q00$ nano app.py
yu@Tokito:~/q00$ ruff check .
invalid-syntax: Simple statements must be separated by newlines or semicolons
--> app.py:5:32
3 | from math_utils import calculate_total
4 |
5 | print(calculate_total(12.5, 4))from math_utils import calculate_total
6 |
7 | print(calculate_total(12.5, 4))

Found 1 error.

```

图 5: 修改函数名称并排查语法错误

```

yu@Tokito:~/q06$ nano app.py
yu@Tokito:~/q06$ ruff check .
All checks passed!
yu@Tokito:~/q06$ python3 -m pytest -q
.
1 passed in 0.03s
yu@Tokito:~/q06$ python3 app.py
50.0
yu@Tokito:~/q06$ git status --short
fatal: not a git repository (or any of the parent directories): .git
yu@Tokito:~/q06$ find ~ -type d -name .git 2>/dev/null
/home/yu/sysdev/q03/.git
yu@Tokito:~/q06$ git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
Initialized empty Git repository in /home/yu/q06/.git/
yu@Tokito:~/q06$ git branch -M main
yu@Tokito:~/q06$ git status --short
?? __pycache__/
?? app.py
?? math_utils.py
?? test_math_utils.py
yu@Tokito:~/q06$ cat > .gitignore <<'EOF'
> __pycache__/
> .pytest_cache/
> *.py[cod]
> EOF
yu@Tokito:~/q06$ git status --short
?? .gitignore
?? app.py
?? math_utils.py
?? test_math_utils.py
yu@Tokito:~/q06$ git add .gitignore
yu@Tokito:~/q06$ git commit -m "chore: ignore Python cache files"
[main (root-commit) 91367ac] chore: ignore Python cache files
1 file changed, 3 insertions(+)
create mode 100644 .gitignore
yu@Tokito:~/q06$ git add math_utils.py
yu@Tokito:~/q06$ git commit -m "feat: add calculate_total utility"
[main 6bca5b3] feat: add calculate_total utility
1 file changed, 2 insertions(+)
create mode 100644 math_utils.py
yu@Tokito:~/q06$ git add app.py test_math_utils.py
yu@Tokito:~/q06$ git commit -m "test: add calculate_total usage and test"
[main e665c9d] test: add calculate_total usage and test
2 files changed, 9 insertions(+)
create mode 100644 app.py
create mode 100644 test_math_utils.py

```

图 6: 初始化 Git 仓库并完成分阶段提交

```

yu@Tokito:~/q06$ ruff check .
All checks passed!
yu@Tokito:~/q06$ python3 -m pytest -q
.
1 passed in 0.03s
yu@Tokito:~/q06$ python3 app.py
50.0
yu@Tokito:~/q06$ git log --oneline --decorate -3
e665c9d (HEAD -> main) test: add calculate_total usage and test
6bca5b3 feat: add calculate_total utility
91367ac chore: ignore Python cache files
yu@Tokito:~/q06$ git status
On branch main
nothing to commit, working tree clean
yu@Tokito:~/q06$ git remote -v
origin https://github.com/Tokito-git/sdt_week2-25060012017.git (fetch)
origin https://github.com/Tokito-git/sdt_week2-25060012017.git (push)
yu@Tokito:~/q06$

```

图 7: 代码检查、单元测试和程序运行结果

实验结果：本实验完成了一个简单的 Python 计算项目，实现了 ‘calculate_total’ 函数，并通过程序调用验证其输出结果为 50.0。pytest 测试结果为 ‘1 passed’，Ruff 检查

结果为 ‘All checks passed’, 说明当前代码能够正常运行并符合基本代码规范。

实验同时使用 Git 完成了三次分阶段提交, 提交内容分别对应缓存文件忽略配置、核心功能函数以及程序入口和测试代码。最终 Git 工作区状态为 clean, 满足实验对代码管理和提交记录的要求。

实验体会: 通过本次实验, 我认识到程序能够运行并不代表代码质量已经合格。pytest 可以验证功能是否正确, Ruff 可以发现代码规范问题, 而 Git 分阶段提交则能够清晰记录开发过程。以后进行科研实验或深度学习项目开发时, 测试、代码检查和版本管理都能够帮助快速定位问题, 减少重复调试的时间。

注意: Python 项目中应将 ‘_pycache_’、‘.pytest.cache’ 等自动生成目录写入 ‘.gitignore’, 避免将无关缓存文件提交到仓库。代码修改后应依次运行程序、pytest 和 Ruff 检查, 确认结果正确后再进行 Git 提交。实验必须分阶段提交, 不能最后一次性提交全部文件, 并应在实验报告中记录各次 commit 的作用。

3.9 实验七: 使用 pdb、pytest 和 Git 调试归并排序程序

实验目的: 学习使用 Python 调试器 pdb 定位程序错误, 掌握归并排序算法的实现与修正方法, 理解单元测试在验证程序正确性中的作用, 并通过 Git 分阶段提交记录代码修改过程。

创建实验目录并进入 q07 目录:

```
1 cd ~/sdt-week2/q07
```

查看并修改归并排序程序 merge_sort.py。实验初始阶段, 程序存在缩进错误、语法错误和归并过程中的下标错误。第一次运行时, 终端报错如下:

```
1 IndentationError: expected an indented block after function definition
2 SyntaxError: expected ':'
```

根据错误提示检查代码后, 继续运行程序并进入 pdb 调试模式, 观察归并过程中左右子数组和下标变量的变化情况:

```
1 python3 merge_sort.py
```

在 pdb 中依次查看变量:

```
1 p left
2 p right
3 p i
4 p j
5 p left[i]
6 p right[j]
```

调试过程中发现，归并操作里将右侧数组元素写成了 `right[i]`，正确写法应为 `right[j]`。这属于典型的循环下标错误，会导致归并结果异常。根据调试信息修正代码后，程序能够正常运行。

修正后的关键代码如下：

```
1 def merge(left, right):
2     result = []
3     i = 0
4     j = 0
5     while i < len(left) and j < len(right):
6         if left[i] <= right[j]:
7             result.append(left[i])
8             i += 1
9         else:
10            result.append(right[j])
11            j += 1
```

继续运行程序：

```
1 python3 merge_sort.py
```

程序输出结果如下：

```
1 [1, 1, 2, 3, 4, 5, 6, 9]
```

这说明归并排序程序已经能够正确输出排序结果。

创建并运行单元测试，使用 `pytest` 验证归并排序功能：

```
1 python3 -m pytest -v
```

测试结果如下：

```
1 collected 2 items
2
3 test_merge_sort.py::test_normal_input PASSED
4 test_merge_sort.py::test_duplicate_elements PASSED
5
6 2 passed in 0.06s
```

测试结果显示两个测试用例均通过，说明程序在普通输入和重复元素输入两种情况下都能正确运行。

创建 Git 仓库并进行分阶段提交：

```
1 cd ~/sdt-week2
2 git init
```

```

3 git add q07/merge_sort.py
4 git commit -m "fix: correct merge sort index"
5 git add q07/test_merge_sort.py
6 git commit -m "test: add merge sort cases"
7 git log --oneline

```

提交历史示例如下：

```

1 2b4f479 (HEAD -> master) test: add merge sort cases
2 f680441 fix: correct merge sort index

```

这说明本实验按照“先修复代码、再补充测试”的顺序进行了分阶段提交，没有将所有文件一次性提交。

```

yu@Tokito:~$ cd ~/sdt-week2/q07
yu@Tokito:~/sdt-week2/q07$
yu@Tokito:~/sdt-week2/q07$ nano merge_sort.py
yu@Tokito:~/sdt-week2/q07$ python3 merge_sort.py
File "/home/yu/sdt-week2/q07/merge_sort.py", line 2
result = []
^
IndentationError: expected an indented block after function definition on line 1
yu@Tokito:~/sdt-week2/q07$ nano merge_sort.py
yu@Tokito:~/sdt-week2/q07$ python3 merge_sort.py
File "/home/yu/sdt-week2/q07/merge_sort.py", line 4
result = []
^
IndentationError: expected an indented block after function definition on line 3
yu@Tokito:~/sdt-week2/q07$ nano merge_sort.py
yu@Tokito:~/sdt-week2/q07$ python3 merge_sort.py
> /home/yu/sdt-week2/q07/merge_sort.py(10)merge()
-> if left[i] <= right[j]:
(Pdb) p left
[3]
(Pdb) p right
[1]
(Pdb) p i
0
(Pdb) p j
0
(Pdb) p left[i]
3
(Pdb) p right[j]
1
(Pdb) n
> /home/yu/sdt-week2/q07/merge_sort.py(14)merge()
-> result.append(right[i])
(Pdb) --KeyboardInterrupt--
(Pdb) q
Traceback (most recent call last):
  File "/home/yu/sdt-week2/q07/merge_sort.py", line 26, in <module>
    print(merge_sort([3, 1, 4, 1, 5, 9, 2, 6]))
  File "/home/yu/sdt-week2/q07/merge_sort.py", line 24, in merge_sort
    return merge(merge_sort(a[:m]), merge_sort(a[m:]))
  File "/home/yu/sdt-week2/q07/merge_sort.py", line 24, in merge_sort
    return merge(merge_sort(a[:m]), merge_sort(a[m:]))
  File "/home/yu/sdt-week2/q07/merge_sort.py", line 24, in merge_sort
    return merge(merge_sort(a[:m]), merge_sort(a[m:]))
  File "/home/yu/sdt-week2/q07/merge_sort.py", line 14, in merge
    result.append(right[i])
  File "/home/yu/sdt-week2/q07/merge_sort.py", line 14, in merge
    result.append(right[i])
  File "/usr/lib/python3.10/bdb.py", line 90, in trace_dispatch
    return self.dispatch_line(frame)
  File "/usr/lib/python3.10/bdb.py", line 115, in dispatch_line
    if self.quitting: raise BdbQuit
bdb.BdbQuit
yu@Tokito:~/sdt-week2/q07$

```

图 8: 使用 pdb 调试归并排序程序并定位下标错误


```
Ctrl-C: KeyboardInterrupt: False KeyboardInterrupt
bdb.BdbQuit
yu@Tokito:~/sdt-week2/q07$ nano merge_sort.py
yu@Tokito:~/sdt-week2/q07$ python3 merge_sort.py
File "/home/yu/sdt-week2/q07/merge_sort.py", line 5
    while i < len(left) and j < len(right)
                                     ^
SyntaxError: expected ':'
yu@Tokito:~/sdt-week2/q07$ nano merge_sort.py
yu@Tokito:~/sdt-week2/q07$ python3 merge_sort.py
yu@Tokito:~/sdt-week2/q07$ python3 merge_sort.py
yu@Tokito:~/sdt-week2/q07$ nano merge_sort.py
yu@Tokito:~/sdt-week2/q07$ python3 merge_sort.py
[1, 1, 2, 3, 4, 5, 6, 9]
```

图 9: 修正归并排序代码中的下标与语法问题

```
yu@Tokito:~/sdt-week2/q07$ python3 -m pytest --version
pytest 6.2.5
yu@Tokito:~/sdt-week2/q07$ python3 -m pytest -v
===== test session starts =====
platform linux -- Python 3.10.12, pytest-6.2.5, py-1.10.0, pluggy-0.13.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /home/yu/sdt-week2/q07
collected 2 items

test_merge_sort.py::test_normal_input PASSED [ 50%]
test_merge_sort.py::test_duplicate_elements PASSED [100%]

===== 2 passed in 0.06s =====
```

图 10: 运行 pytest 验证归并排序功能正确性

```
yu@Tokito:~/sdt-week2/q07$ python3 -m pytest --version
pytest 6.2.5
yu@Tokito:~/sdt-week2/q07$ python3 -m pytest -v
===== test session starts =====
platform linux -- Python 3.10.12, pytest-6.2.5, py-1.10.0, pluggy-0.13.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /home/yu/sdt-week2/q07
collected 2 items

test_merge_sort.py::test_normal_input PASSED [ 50%]
test_merge_sort.py::test_duplicate_elements PASSED [100%]

===== 2 passed in 0.06s =====
```

图 11: 初始化 Git 仓库并完成分阶段提交

实验结果：本实验完成了归并排序程序的调试与修正，最终程序能够正确输出排序结果 [1, 1, 2, 3, 4, 5, 6, 9]。pytest 测试结果为 2 passed，说明程序在不同输入条件下都能正确运行。Git 提交记录清晰展示了代码修复和测试补充的过程，符合实验对分阶段提交的要求。

```
1680441 fix: correct merge sort index
yu@Tokito:~/sdt-week2$ cd ~/sdt-week2/q07
yu@Tokito:~/sdt-week2/q07$ python3 merge_sort.py
[1, 1, 2, 3, 4, 5, 6, 9]
yu@Tokito:~/sdt-week2/q07$ python3 -m pytest -v
===== test session starts =====
platform linux -- Python 3.10.12, pytest-6.2.5, py-1.10.0, pluggy-0.13.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /home/yu/sdt-week2/q07
collected 2 items

test_merge_sort.py::test_normal_input PASSED [ 50%]
test_merge_sort.py::test_duplicate_elements PASSED [100%]

===== 2 passed in 0.05s =====
```

图 12: 归并排序程序运行结果与测试结果

实验体会：本次实验中，缩进错误、语法错误和下标错误都属于开发中常见的问题。pdb 的作用不是直接替你改代码，而是帮助你在运行时确认变量状态，从而定位错误来源。以后做科研代码或深度学习项目时，归并、索引、循环边界这类问题仍然会反复出现，养成先看错误信息、再看变量状态、最后再改代码的习惯，可以显著减少排错时间。

注意：实验报告中的截图应按文件名规范保存到 screenshots/q07/ 目录下，提交前应确认程序运行、pytest 测试和 Git 提交记录都已完成。实验必须分阶段提交，commit 信息要能反映每次修改的作用，并在报告中写清楚每次提交的内容。

3.10 实验八：使用 cProfile 分析并优化 Python 程序性能

实验目的：掌握 Python 程序性能分析的基本方法，学习使用 time 命令测量程序运行时间，使用 cProfile 定位性能瓶颈，并通过合理选择数据结构优化程序性能。同时，通过 Git 分阶段提交记录程序的开发和优化过程。

创建实验目录：

```
1 mkdir q08
2 cd q08
```

创建随机单词数据生成程序 generate_words.py:

```
1 nano generate_words.py
```

在文件中输入以下内容：

```
1 import random
2 import string
3
4 random.seed(42)
5
6 vocabulary = [
7     ''.join(random.choices(string.ascii_lowercase, k=5))
8     for _ in range(1000)
9 ]
10
11 with open("words.txt", "w", encoding="utf-8") as file:
12     for _ in range(30000):
13         file.write(random.choice(vocabulary) + "\n")
```

运行数据生成程序：

```
1 python3 generate_words.py
2 wc -l words.txt
```

```
3 head words.txt
```

终端输出结果如下：

```
1 30000 words.txt
```

这说明程序已经成功生成了包含 30000 个单词的数据文件。数据中的单词来自一个包含 1000 个不同单词的词汇表，因此后续统计结果应当为 1000 个不同单词。

实验过程截图如图 13 所示。

```

yu@Tokito: ~/sdt-week2/q08
yu@Tokito:~$ cd ~/sdt-week2
yu@Tokito:~/sdt-week2$ cd q08
yu@Tokito:~/sdt-week2/q08$ cat > generate_words.py <<'PY'
> import random

random.seed(20260831)
vocab = [f"w{i}" for i in range(1000)]

with open("words.txt", "w", encoding="utf-8") as f:
    f.write(" ".join(random.choice(vocab) for _ in range(30000)))
> PY
yu@Tokito:~/sdt-week2/q08$ ls -l generate_words.py
-rw-rw-r-- 1 yu yu 195  9月  6 11:33 generate_words.py
yu@Tokito:~/sdt-week2/q08$ python3 generate_words.py
yu@Tokito:~/sdt-week2/q08$ ls -lh words.txt
-rw-rw-r-- 1 yu yu 144K  9月  6 11:33 words.txt
yu@Tokito:~/sdt-week2/q08$ wc -w words.txt
30000 words.txt
yu@Tokito:~/sdt-week2/q08$ head -c 100 words.txt
echo
w204 w193 w923 w525 w302 w465 w689 w80 w451 w575 w786 w806 w844 w698 w932 w579 w762 w417 w859 w772 w
yu@Tokito:~/sdt-week2/q08$ cat > wordfreq.py <<'PY'
> words = open("words.txt", encoding="utf-8").read().split()
unique = []
for word in words:
    if word not in unique:
        unique.append(word)
print("count=", len(unique))
> PY
yu@Tokito:~/sdt-week2/q08$ python3 wordfreq.py
count= 1000

```

图 13: 生成 30000 条随机单词测试数据

创建初始版本的单词去重程序 wordfreq.py:

```
1 nano wordfreq.py
```

在文件中输入以下内容：

```

1 words = open("words.txt", encoding="utf-8").read().split()
2
3 unique = []
4 for word in words:
5     if word not in unique:
6         unique.append(word)
7
8 print("count=", len(unique))

```

该程序首先读取 words.txt 中的全部单词，然后使用列表保存已经发现的不同单词。每处理一个新单词时，程序通过 word not in unique 判断该单词是否已经存在于列表中。

运行初始程序：

```
1 python3 wordfreq.py
```

程序输出结果如下：

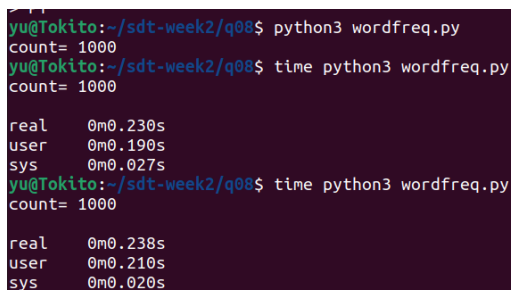
```
1 count= 1000
```

输出结果表明，程序能够正确统计出 1000 个不同单词。

使用 `time` 命令测量初始程序的运行时间：

```
1 time python3 wordfreq.py
2 time python3 wordfreq.py
```

通过连续运行两次程序，可以减少单次运行误差，并记录初始版本的 `real` 运行时间。实验运行结果截图如图 14 所示。



```
yu@Tokito:~/sdt-week2/q08$ python3 wordfreq.py
count= 1000
yu@Tokito:~/sdt-week2/q08$ time python3 wordfreq.py
count= 1000

real    0m0.230s
user    0m0.190s
sys     0m0.027s
yu@Tokito:~/sdt-week2/q08$ time python3 wordfreq.py
count= 1000

real    0m0.238s
user    0m0.210s
sys     0m0.020s
```

图 14: 测量列表版本程序的运行时间

使用 `cProfile` 对初始程序进行性能分析：

```
1 python3 -m cProfile -s cumulative wordfreq.py
```

`cProfile` 会统计函数调用次数以及各函数消耗的运行时间。分析结果中，列表成员判断语句：

```
1 if word not in unique:
```

是影响程序性能的主要原因。列表查找需要从头到尾逐个比较元素，随着 `unique` 列表长度增加，查找所需的时间也会增加。

初始程序中，假设单词总数为 n ，不同单词数量为 m ，列表查找的时间复杂度约为：

```
1 O(nm)
```

当输入数据规模较大时，列表线性查找会成为明显的性能瓶颈。

性能分析结果截图如图 15 所示。

```

sys 0.000000
yu@Tokito:~/sdt-week2/q00$ python3 -m cProfile -s cumulative wordfreq.py
count= 1000
1012 function calls in 0.214 seconds

Ordered by: cumulative time

ncalls  tottime  percall  cumtime  percall filename:lineno(function)
1      0.000    0.000    0.214    0.214 {built-in method builtins.exec}
1      0.173    0.173    0.214    0.214 wordfreq.py:1(<module>)
1000    0.023    0.000    0.023    0.000 {method 'append' of 'list' objects}
1      0.017    0.017    0.017    0.017 {method 'split' of 'str' objects}
1      0.000    0.000    0.001    0.001 {method 'read' of '_io.TextIOWrapper' objects}
1      0.000    0.000    0.001    0.001 codecs.py:319(decode)
1      0.001    0.001    0.001    0.001 {built-in method _codecs.utf_8_decode}
1      0.000    0.000    0.000    0.000 {built-in method io.open}
1      0.000    0.000    0.000    0.000 {built-in method builtins.print}
1      0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}
1      0.000    0.000    0.000    0.000 codecs.py:309(__init__)
1      0.000    0.000    0.000    0.000 codecs.py:260(__init__)
1      0.000    0.000    0.000    0.000 {built-in method builtins.len}

```

图 15: 使用 cProfile 定位程序性能瓶颈

分析性能瓶颈后，将列表数据结构替换为集合。集合底层使用哈希表，通常可以在常数时间内完成成员判断和插入操作。

修改 wordfreq.py:

```
1 nano wordfreq.py
```

将程序修改为:

```

1 words = open("words.txt", encoding="utf-8").read().split()
2
3 unique = set()
4 for word in words:
5     unique.add(word)
6
7 print("count=", len(unique))

```

优化后的程序使用 `set` 保存不同单词。集合中的元素不能重复，因此不需要手动进行列表成员判断，也能够直接完成去重操作。

优化程序后运行检查:

```
1 python3 wordfreq.py
```

程序输出如下:

```
1 count= 1000
```

优化前后程序的输出结果相同，说明修改数据结构没有改变程序的功能。

优化代码的过程截图如图 16 所示。



```
GNU nano 6.2 wordfreq.py *
words = open("words.txt", encoding="utf-8").read().split()
unique = set()
for word in words:
    unique.add(word)
print("count=", len(unique))
```

图 16: 使用集合替换列表完成程序优化

再次使用 `time` 命令测量优化后的程序:

```
1 time python3 wordfreq.py
2 time python3 wordfreq.py
```

记录优化后的两次运行时间, 并将其与优化前的运行时间进行比较。加速比可以按照以下公式计算:

加速比的计算公式为:

$$\text{加速比} = \frac{\text{优化前运行时间中位数}}{\text{优化后运行时间中位数}}$$

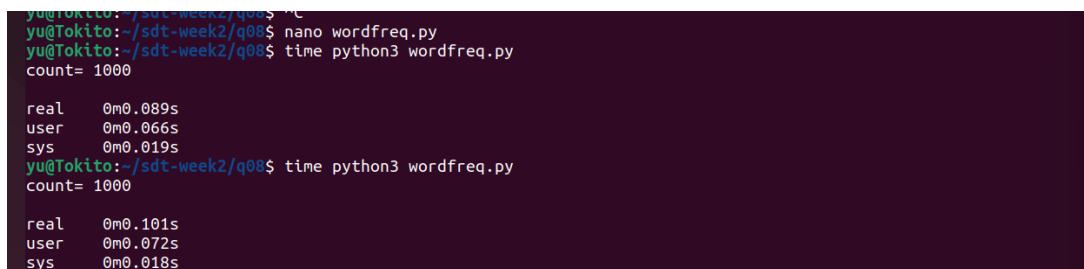
由于集合的成员判断和插入操作平均时间复杂度为 $O(1)$, 优化后程序的总体时间复杂度约为:

优化后的程序总体时间复杂度约为:

$$O(n)$$

因此, 随着输入数据规模增加, 使用集合的程序通常比使用列表的程序具有更好的性能。

优化后程序的运行结果截图如图 17 所示。



```
yu@Tokito: ~/sdt-week2/q08$ nano wordfreq.py
yu@Tokito:~/sdt-week2/q08$ time python3 wordfreq.py
count= 1000

real    0m0.089s
user    0m0.066s
sys     0m0.019s
yu@Tokito:~/sdt-week2/q08$ time python3 wordfreq.py
count= 1000

real    0m0.101s
user    0m0.072s
sys     0m0.018s
```

图 17: 测量集合版本程序的运行时间

本实验使用 Git 对程序开发过程进行分阶段提交。首先查看当前仓库状态:

```
1 cd ~/sdt-week2
2 git status --short
```

第一次提交数据生成程序和测试数据：

```
1 git add q08/generate_words.py q08/words.txt
2 git commit -m "data: generate word frequency dataset"
```

第二次提交优化后的单词统计程序：

```
1 git add q08/wordfreq.py
2 git commit -m "perf: optimize word frequency with set"
```

查看 q08 实验的提交历史：

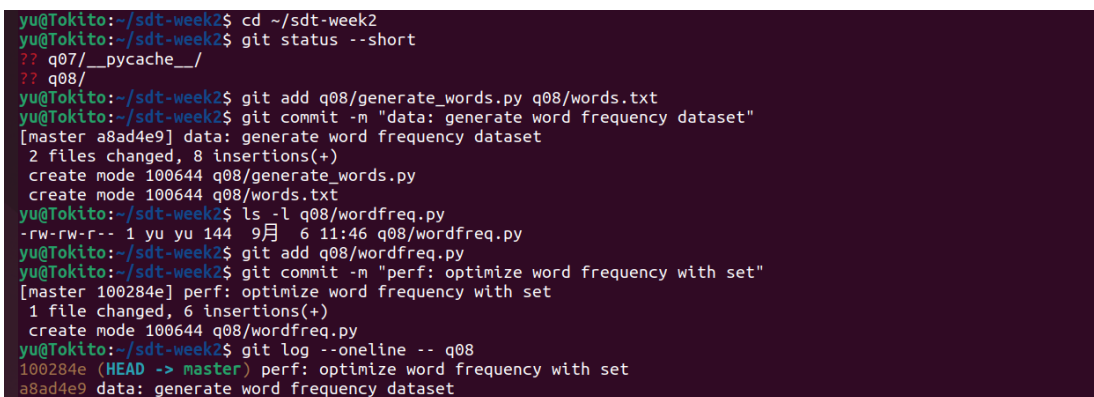
```
1 git log --oneline -- q08
```

提交历史示例如下：

```
1 100284e (HEAD -> master) perf: optimize word frequency with set
2 a8ad4e9 data: generate word frequency dataset
```

实际提交编号以终端中显示的结果为准。上述提交记录表明，本实验至少进行了两次独立提交：第一次提交数据生成程序和测试数据，第二次提交性能优化后的程序，符合实验要求的分阶段提交规范。

Git 提交历史截图如图 18 所示。



```
yu@Tokito:~/sdt-week2$ cd ~/sdt-week2
yu@Tokito:~/sdt-week2$ git status --short
?? q07/__pycache__/
?? q08/
yu@Tokito:~/sdt-week2$ git add q08/generate_words.py q08/words.txt
yu@Tokito:~/sdt-week2$ git commit -m "data: generate word frequency dataset"
[master a8ad4e9] data: generate word frequency dataset
2 files changed, 8 insertions(+)
create mode 100644 q08/generate_words.py
create mode 100644 q08/words.txt
yu@Tokito:~/sdt-week2$ ls -l q08/wordfreq.py
-rw-rw-r-- 1 yu yu 144  9月  6 11:46 q08/wordfreq.py
yu@Tokito:~/sdt-week2$ git add q08/wordfreq.py
yu@Tokito:~/sdt-week2$ git commit -m "perf: optimize word frequency with set"
[master 100284e] perf: optimize word frequency with set
1 file changed, 6 insertions(+)
create mode 100644 q08/wordfreq.py
yu@Tokito:~/sdt-week2$ git log --oneline -- q08
100284e (HEAD -> master) perf: optimize word frequency with set
a8ad4e9 data: generate word frequency dataset
```

图 18: 查看实验八 Git 分阶段提交记录

实验结果：本实验生成了包含 30000 个单词的测试数据，并编写了单词去重统计程序。初始版本使用列表进行成员判断，程序输出结果为 count=1000。通过 time 命令测量运行时间，并使用 cProfile 分析程序性能后，确定列表线性查找是主要性能瓶颈。

随后将列表替换为集合，优化后的程序仍然输出 1000 个不同单词，说明程序功能保持不变。由于集合的成员判断和插入操作平均时间复杂度为 $O(1)$ ，优化后的程序运行效率得到提升。实验还使用 Git 完成了数据生成阶段和程序优化阶段的分阶段提交，提交历史能够清晰记录程序的开发过程。

实验体会：通过本次实验，我认识到程序能够得到正确结果并不代表程序具有良好的性能。对于运行速度较慢的程序，应先使用 `time` 测量运行时间，再使用 `cProfile` 定位具体的性能瓶颈，不能仅凭主观判断进行优化。

本实验中，列表的线性查找导致程序在数据规模增大时运行效率下降，而集合能够利用哈希表快速完成成员判断。这个例子说明，合理选择数据结构对程序性能有直接影响。以后进行科研计算或深度学习项目开发时，数据预处理、样本去重和缓存查询等操作也经常会遇到类似问题。掌握性能分析和数据结构优化方法，可以减少程序运行时间，提高实验效率。

注意：实验过程中应保留原始程序和优化程序的运行结果，并在实验报告中记录优化前后的运行时间以及加速比。使用 `cProfile` 时应重点关注累计耗时较高的函数和语句。实验文件必须上传到 GitHub 或 Gitee，不能只进行一次全量提交。

Git 提交命令必须分开执行。例如：

```
1 git add q08/wordfreq.py
2 git commit -m "perf: optimize word frequency with set"
```

不能将两条命令直接拼接为一行，否则 Git 会把后面的参数错误地解释为 `git add` 的选项。

4 课后练习

本部分记录课后完成的练习。每个练习至少写明练习名称、关键命令或代码、运行结果和个人体会。以下表格仅为填写格式，请替换为真实内容。

表 2: 课后练习记录

编号	练习名称	关键内容与结果	截图
1	进程替换与环境变量比较	使用 <code>diff <(printenv sort)</code> 与 <code><(export sort)</code> 比较环境变量的值与 Shell 导出声明格式，理解进程替换的使用方法。	有
2	后台任务管理	使用 <code>Ctrl-Z</code> 挂起 <code>sleep 10000</code> ，使用 <code>bg</code> 恢复后台运行，再用 <code>pgrep</code> 和 <code>pkill</code> 查找并终止任务。	有
3	进程执行顺序控制	使用 <code>wait</code> 或命令连接符，保证前一个进程执行结束后再启动后一个进程。	有
4	Shell alias 配置	创建 <code>dc</code> 别名，使输入 <code>dc</code> 时能够执行 <code>cd</code> ，并验证别名的临时配置效果。	有
5	常用命令统计	使用 <code>history</code> 、 <code>awk</code> 、 <code>sort</code> 和 <code>uniq</code> 统计最常用的十条命令，并分析可配置的快捷别名。	有
6	Dotfiles 版本控制	创建 <code>~/dotfiles</code> 目录，将 Shell 配置文件纳入 Git 版本控制，并查看提交历史。	有
7	Shell 个性化配置	修改 <code>\$PS1</code> 配置 Shell 提示符，观察配置对终端显示效果的影响。	有
8	Dotfiles 自动安装	编写安装脚本，通过符号链接将配置文件自动部署到新机器的用户目录。	有
9	使用 <code>wait</code> 控制任务顺序	启动 <code>sleep 60 &</code> ，使用 <code>wait</code> 等待后台任务结束，再执行 <code>ls</code> ，理解进程同步。	有
10	合并排序调试	使用 Python 和 <code>pdb</code> 调试合并排序，定位 <code>merge</code> 函数中错误使用 <code>right[i]</code> 的问题，并修复为 <code>right[j]</code> 。	有

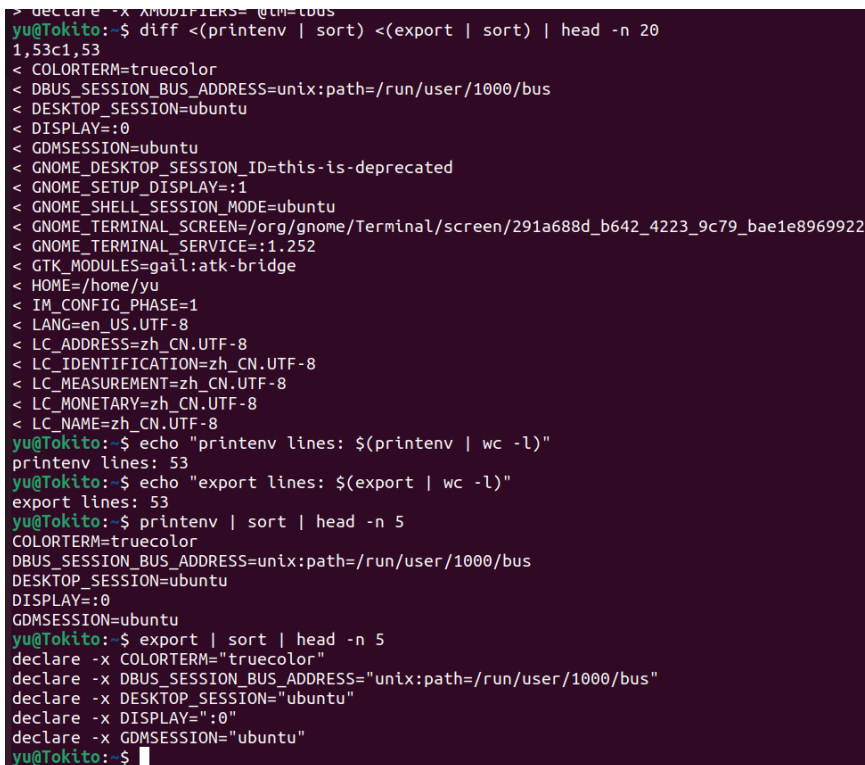
5 课后练习

5.1 练习1: ls -l

做法：执行以下命令查看根目录的详细信息。

```
1 ls -l /
```

注意：第一个字符表示文件类型，后面九个字符表示所有者、所属组和其他用户的权限。



```
> declare -x XMODIFIERS=@xmodrm=tc00s
yu@Tokito:~$ diff <(printenv | sort) <(export | sort) | head -n 20
1,53c1,53
< COLORTERM=truecolor
< DBUS_SESSION_BUS_ADDRESS=unix:path=/run/user/1000/bus
< DESKTOP_SESSION=ubuntu
< DISPLAY=:0
< GDMSESSION=ubuntu
< GNOME_DESKTOP_SESSION_ID=this-is-deprecated
< GNOME_SETUP_DISPLAY=:1
< GNOME_SHELL_SESSION_MODE=ubuntu
< GNOME_TERMINAL_SCREEN=/org/gnome/Terminal/screen/291a688d_b642_4223_9c79_bae1e8969922
< GNOME_TERMINAL_SERVICE=:1.252
< GTK_MODULES=gail:atk-bridge
< HOME=/home/yu
< IM_CONFIG_PHASE=1
< LANG=en_US.UTF-8
< LC_ADDRESS=zh_CN.UTF-8
< LC_IDENTIFICATION=zh_CN.UTF-8
< LC_MEASUREMENT=zh_CN.UTF-8
< LC_MONETARY=zh_CN.UTF-8
< LC_NAME=zh_CN.UTF-8
yu@Tokito:~$ echo "printenv lines: $(printenv | wc -l)"
printenv lines: 53
yu@Tokito:~$ echo "export lines: $(export | wc -l)"
export lines: 53
yu@Tokito:~$ printenv | sort | head -n 5
COLORTERM=truecolor
DBUS_SESSION_BUS_ADDRESS=unix:path=/run/user/1000/bus
DESKTOP_SESSION=ubuntu
DISPLAY=:0
GDMSESSION=ubuntu
yu@Tokito:~$ export | sort | head -n 5
declare -x COLORTERM="truecolor"
declare -x DBUS_SESSION_BUS_ADDRESS="unix:path=/run/user/1000/bus"
declare -x DESKTOP_SESSION="ubuntu"
declare -x DISPLAY=":0"
declare -x GDMSESSION="ubuntu"
yu@Tokito:~$
```

图 19: 练习1: 使用 ls -l 查看文件信息

5.2 练习2: Glob 通配模式

做法：创建多个测试文件，使用不同的通配符进行匹配。

```
1 touch a.txt b.txt c.txt file1.txt file2.txt file10.txt
2 ls *.txt
3 ls file?.txt
4 ls {a,b,c}.txt
```

注意：* 匹配任意长度字符，? 只匹配一个字符，所以 file?.txt 不会匹配 file10.txt。

```
yu@Tokito:~$ sleep 10000
^Z
[1]+  Stopped                  sleep 10000
yu@Tokito:~$ bg
[1]+  sleep 10000 &
yu@Tokito:~$ jobs
[1]+  Running                  sleep 10000 &
yu@Tokito:~$ pgrep -af 'sleep 10000'
20022 sleep 10000
yu@Tokito:~$ pkill -TERM -F '^sleep 10000$'
pkill: pidfile not valid
Try 'pkill --help' for more information.
yu@Tokito:~$ pkill -TERM -f '^sleep 10000$'
[1]+  Terminated              sleep 10000
yu@Tokito:~$ pgrep -af 'sleep 10000'
yu@Tokito:~$
```

图 20: 练习2: Glob 通配模式

5.3 练习3: 使用命令连接符控制执行顺序

做法: 使用 `&&`、`||` 和分号控制多个命令的执行顺序。

```
1 echo "first command" && echo "second command"
2 echo "success" || echo "failed"
3 echo "command A"; echo "command B"
```

为了观察命令执行失败时的不同效果, 执行:

```
1 false && echo "this will not be printed"
2 false || echo "the previous command failed"
3 false; echo "this command will still be executed"
```

结果分析: `&&` 表示当前一个命令执行成功时, 才执行后一个命令; `||` 表示当前一个命令执行失败时, 才执行后一个命令; 分号表示依次执行命令, 不关注前一个命令的返回值。这些连接符可以用于编写 Shell 脚本中的条件控制逻辑。

```
sleep has finished
yu@Tokito:~$ sleep 3 &
[1] 20153
yu@Tokito:~$ pid=$!
[1]+  Done                  sleep 3
yu@Tokito:~$ echo "started pid=$pid"
started pid=20153
yu@Tokito:~$ wait "$pid"
yu@Tokito:~$ echo "finished"
finished
yu@Tokito:~$
```

图 21: 练习3: 使用命令连接符控制执行顺序

5.4 练习4: 创建和使用 Shell 别名

做法: 为常用命令设置别名, 并验证别名的执行效果。

```
1 alias gs='git status'
2 alias ga='git add'
```

```
3 alias ll='ls -a1F'
4 alias py='python3'
```

使用以下命令检查别名：

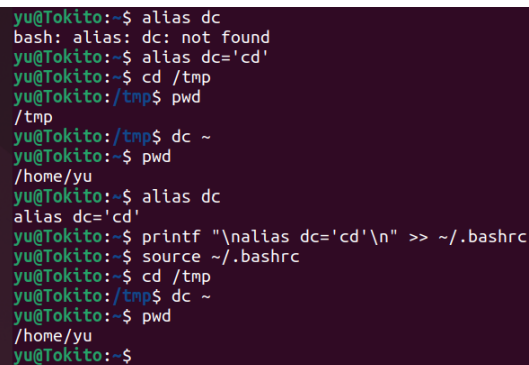
```
1 alias gs
2 alias ga
3 alias ll
4 alias py
5 type gs
6 type py
```

分别测试别名：

```
1 gs
2 ll
3 py --version
```

结果分析：通过 `alias` 可以为较长或使用频率较高的命令设置简短名称。例如，`gs` 等价于 `git status`，`py` 等价于 `python3`。使用 `type` 可以检查当前命令是否为别名。

需要注意，直接在终端中设置的别名只对当前 Shell 会话有效。若要永久保存，应将配置写入 `~/.bashrc`，然后执行 `source ~/.bashrc`。



```
yu@Tokito:~$ alias dc
bash: alias: dc: not found
yu@Tokito:~$ alias dc='cd'
yu@Tokito:~$ cd /tmp
yu@Tokito:/tmp$ pwd
/tmp
yu@Tokito:/tmp$ dc ~
yu@Tokito:~$ pwd
/home/yu
yu@Tokito:~$ alias dc
alias dc='cd'
yu@Tokito:~$ printf "\nalias dc='cd'\n" >> ~/.bashrc
yu@Tokito:~$ source ~/.bashrc
yu@Tokito:~$ cd /tmp
yu@Tokito:/tmp$ dc ~
yu@Tokito:~$ pwd
/home/yu
yu@Tokito:~$
```

图 22: 练习4：创建和验证 Shell 别名

5.5 练习5：统计历史命令的使用频率

做法：使用 `history` 获取历史命令，并结合 `awk`、`sort` 和 `uniq` 统计命令的使用次数。

```
1 history | awk '{ $1="" ; print substr($0,2) }' | sort | uniq -c | sort -n | tail -n 10
```

为了减少命令中的空格影响，也可以使用下面的方式提取命令名称：

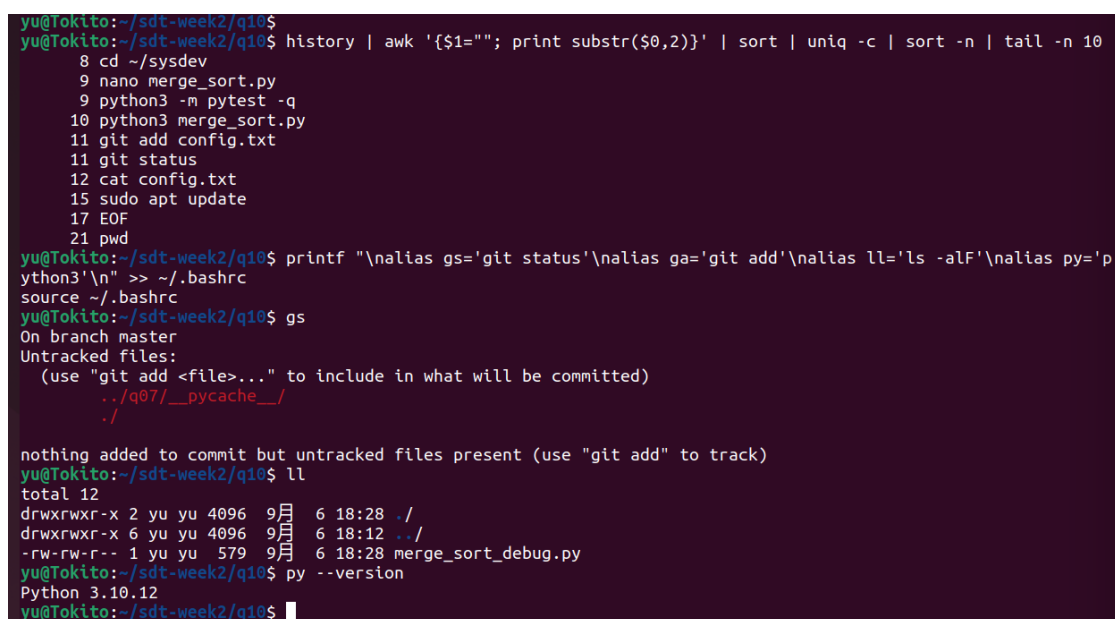
```
1 history | awk '{ $1=""; print $0 }' | sed 's/^ *//' | awk '{ print $1 }' | sort |
    uniq -c | sort -n | tail -n 10
```

根据常用命令配置别名：

```
1 printf "\nalias gs='git status'\nalias ga='git add'\nalias ll='ls -alF'\nalias
    py='python3'\n" >> ~/.bashrc
2 source ~/.bashrc
3 alias gs
4 alias ga
5 alias ll
6 alias py
```

结果分析：history 用于读取历史命令，awk 用于去除历史编号，sort 用于将相同命令排列在一起，uniq -c 用于统计重复次数，最后使用 sort -n 和 tail -n 10 得到出现次数较多的命令。

需要注意，使用 history 统计时应避免把统计命令本身重复计算过多次。配置 alias 后，必须通过 source ~/.bashrc 重新加载配置，否则新别名不会在当前 Shell 中生效。



```
yu@Tokito:~/sdt-week2/q10$
yu@Tokito:~/sdt-week2/q10$ history | awk '{ $1=""; print substr($0,2)}' | sort | uniq -c | sort -n | tail -n 10
  8 cd ~/sysdev
  9 nano merge_sort.py
  9 python3 -m pytest -q
 10 python3 merge_sort.py
 11 git add config.txt
 11 git status
 12 cat config.txt
 15 sudo apt update
 17 EOF
 21 pwd
yu@Tokito:~/sdt-week2/q10$ printf "\nalias gs='git status'\nalias ga='git add'\nalias ll='ls -alF'\nalias py='p
ython3'\n" >> ~/.bashrc
source ~/.bashrc
yu@Tokito:~/sdt-week2/q10$ gs
On branch master
Untracked files:
  (use "git add <file>..." to include in what will be committed)
    ../q07/___pycache___/
    ./

nothing added to commit but untracked files present (use "git add" to track)
yu@Tokito:~/sdt-week2/q10$ ll
total 12
drwxrwxr-x 2 yu yu 4096  9月  6 18:28 ./
drwxrwxr-x 6 yu yu 4096  9月  6 18:12 ../
-rw-rw-r-- 1 yu yu  579  9月  6 18:28 merge_sort_debug.py
yu@Tokito:~/sdt-week2/q10$ py --version
Python 3.10.12
yu@Tokito:~/sdt-week2/q10$
```

图 23: 练习5: 统计历史命令并配置常用别名

5.6 练习6: 使用 Git 管理配置文件

做法：创建 dotfiles 目录，将 Bash 配置文件复制到该目录，并使用 Git 记录配置文件的版本变化。

```
1 mkdir -p ~/dotfiles
2 cd ~/dotfiles
3 cp ~/.bashrc bashrc
4 git init
5 git branch -M main
6 git status
7 git add bashrc
8 git commit -m "chore: add bash configuration"
```

查看提交记录:

```
1 git log --oneline --graph --all
2 git status
```

创建 Git 忽略文件, 避免提交编辑器产生的临时文件:

```
1 printf "*.swp\n*~\n" > .gitignore
2 git add .gitignore
3 git commit -m "chore: add gitignore"
4 git status
```

结果分析: dotfiles 通常指以点号开头的用户配置文件, 例如 .bashrc 和 .gitconfig。将配置文件放入 Git 仓库后, 可以查看每次修改的提交记录, 也可以在不同计算机之间同步配置。

本实验中每次修改都应单独提交, 不能只在实验结束时进行一次全量提交。提交记录需要保留, 并在实验报告中说明。

```

yu@Tokito:~$ mkdir -p ~/dotfiles
yu@Tokito:~$ cd ~/dotfiles
yu@Tokito:~/dotfiles$ cp ~/.bashrc bashrc
yu@Tokito:~/dotfiles$ git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
Initialized empty Git repository in /home/yu/dotfiles/.git/
yu@Tokito:~/dotfiles$ git branch -M main
yu@Tokito:~/dotfiles$ git status --short
?? bashrc
yu@Tokito:~/dotfiles$ git add bashrc
yu@Tokito:~/dotfiles$ git commit -m "chore: add bash configuration"
[main (root-commit) 56fc09e] chore: add bash configuration
 1 file changed, 124 insertions(+)
 create mode 100644 bashrc
yu@Tokito:~/dotfiles$ git log --oneline --graph --all
* 56fc09e (HEAD -> main) chore: add bash configuration
yu@Tokito:~/dotfiles$ printf "*.swp\n*~\n" > .gitignore
yu@Tokito:~/dotfiles$ git add .gitignore
yu@Tokito:~/dotfiles$ git commit -m "chore: ignore editor temporary files"
[main 4615302] chore: ignore editor temporary files
 1 file changed, 2 insertions(+)
 create mode 100644 .gitignore
yu@Tokito:~/dotfiles$

```

图 24: 练习6: 使用 Git 管理 Bash 配置文件

5.7 练习7: 自定义 Shell 提示符

做法: 通过设置 PS1 修改 Bash 提示符, 使提示符显示用户名、主机名和当前工作目录。

```

1 export PS1='\u@\h:\w\$ '
2 echo "$PS1"

```

将配置写入 ~/.bashrc:

```

1 printf "\nexport PS1='\u@\h:\w\$ '\n" >> ~/.bashrc
2 source ~/.bashrc

```

设置带颜色的提示符:

```

1 export PS1='\[\033[01;32m\]\u@\h\[\033[00m\]:\[\033[01;34m\]\w\[\033[00m\]\$ '

```

如果要永久保存带颜色的提示符, 可以编辑配置文件:

```

1 nano ~/.bashrc

```

在文件末尾加入:

```

1 export PS1='\[\033[01;32m\]\u@\h\[\033[00m\]:\[\033[01;34m\]\w\[\033[00m\]\$ '

```

保存后重新加载:

```

1 source ~/.bashrc

```

结果分析：PS1 是 Bash 的一级提示符变量。其中，\u 表示用户名，\h 表示主机名，\w 表示当前工作目录。颜色代码中的 01;32m 表示绿色，01;34m 表示蓝色，00m 用于恢复默认颜色。

在使用 printf 写入配置文件时，必须对反斜杠进行正确转义，否则 \u 可能被 printf 误认为 Unicode 转义符，从而出现 printf: missing unicode digit for \u 错误。

```

yu@Tokito:~/dotfiles$
yu@Tokito:~/dotfiles$ export PS1='\u@\h:\w\$ '
yu@Tokito:~/dotfiles$ printf "\nexport PS1='\\u@\\h:\\w\\$ '\n" >> ~/.bashrc
bash: printf: missing unicode digit for \u
yu@Tokito:~/dotfiles$ source ~/.bashrc
yu@Tokito:~/dotfiles$ echo "$PS1"
\u@\h:\w\$
yu@Tokito:~/dotfiles$ printf "\nexport PS1='\\u@\\h:\\w\\$ '\n" >> ~/dotfiles/bashrc
bash: printf: missing unicode digit for \u
yu@Tokito:~/dotfiles$ cd ~/dotfiles
yu@Tokito:~/dotfiles$ git add bashrc
yu@Tokito:~/dotfiles$ git commit -m "feat: customize bash prompt"
[main 3f234c5] feat: customize bash prompt
1 file changed, 2 insertions(+)
yu@Tokito:~/dotfiles$
yu@Tokito:~/dotfiles$

```

图 25: 练习7: 自定义带颜色的 Shell 提示符

5.8 练习8: 编写 dotfiles 自动安装脚本

做法：在 dotfiles 目录中编写安装脚本，使用符号链接部署 Bash 配置文件。

```

1 cd ~/dotfiles
2 nano install.sh

```

脚本内容如下：

```

1 #!/usr/bin/env bash
2 set -e
3
4 DOTFILES_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
5
6 ln -sf "$DOTFILES_DIR/bashrc" "$HOME/.bashrc"
7
8 echo "dotfiles installed"
9 echo "Run: source ~/.bashrc"

```

为脚本添加执行权限并运行：

```

1 chmod +x install.sh
2 ./install.sh
3 ls -l ~/.bashrc
4 source ~/.bashrc

```


检查符号链接的目标：

```
1 readlink -f ~/.bashrc
```

6 GitHub/Gitee 仓库与提交记录

代码仓库地址：<https://github.com/Tokito-git/sdt-week1-25060012017>

本周实验代码按照 q01、q02、q03、q04 分目录保存。提交过程中采用分步提交方式，分别记录了文件创建、功能实现、错误修复和报告完善等阶段。仓库链接和提交记录截图见图 ??。

7 实验总结

本周实验使我认识到，工具链的正确使用不仅是记住命令，还包括对输入、输出、错误状态和历史记录进行验证。Shell 实验中需要特别注意空格路径、隐藏文件、表头和退出码；Git 实验中要根据提交历史理解冲突根因；LaTeX 实验中则需要同时检查源文件、编译引擎和交叉引用。以后进行科研代码或深度学习实验时，熟练使用这些工具能够提高实验复现、问题定位和协作开发的效率。

8 GitHub/Gitee 仓库与提交记录

代码仓库地址：<https://github.com/Tokito-git/sdt-week1-25060012017>

本周实验代码按照 q01、q02、q03、q04 分目录保存。提交过程中采用分步提交方式，分别记录了文件创建、功能实现、错误修复和报告完善等阶段。仓库链接和提交记录截图见图 ??。

9 实验总结

本周实验使我认识到，工具链的正确使用不仅是记住命令，还包括对输入、输出、错误状态和历史记录进行验证。Shell 实验中需要特别注意空格路径、隐藏文件、表头和退出码；Git 实验中要根据提交历史理解冲突根因；LaTeX 实验中则需要同时检查源文件、编译引擎和交叉引用。以后进行科研代码或深度学习实验时，熟练使用这些工具能够提高实验复现、问题定位和协作开发的效率。